

Day 5 - Facilitation Guide

(Using Multimedia and CSS)

INDEX

- I. Recap of the previous session
- II. Validation and error messages
- III. Audio and Video in HTML5
- IV. Adding multimedia content with HTML5
- V. Introduction to Cascading Style Sheet (CSS)
- VI. Why CSS is needed in a Web site
- VII. Overview of CSS
- VIII. CSS syntax and selectors
- IX. Browser support for CSS
- X. Different categories of CSS with Examples
- XI. Inline CSS

(2 hrs) ILT

I. Recap of the previous session

In our previous session, we had a chance to deep dive into the wonderful world of HTML5. Today, we're going to recap what we've learned so far to reinforce our understanding of the key concepts and features of HTML5.

Let's start with a quick recap of the basics. HTML, which stands for HyperText Markup Language, is the standard language used to create Web pages. It provides the structure for content on the Web and is the backbone of every Web page you visit. HTML5, the fifth version of HTML, brought several new features and improvements over its predecessors.

Here's a brief overview of the major features of HTML5 :

Simplified Doctype Declaration: Use of `<!DOCTYPE html>` declaration, makes it easier to start a new HTML document.

Semantic Elements: HTML5 introduced semantic elements like `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, and `<footer>`.

New Form Elements: HTML5 introduced new form input types such as `<input type="email">`, `<input type="url">`, `<input type="tel">`, and `<input type="date">` for enhanced user input.

Multimedia Support: HTML5 brought native support for audio and video elements (`<audio>` and `<video>`), allowing seamless integration of multimedia content without relying on third-party plugins like Flash.

Canvas and SVG Graphics: HTML5 introduced the `<canvas>` element for rendering graphics and animations directly within the browser. It also supports Scalable Vector Graphics (SVG) for high-quality, resolution-independent graphics.

There are many other features of HTML5 and we explained all the features with practical examples during our sessions, including creating forms and using semantic elements.

In this session we will have further discussion on HTML5 explaining how to work on embedding multimedia in an HTML page. We will also discuss another important item in building user interfaces ie CSS, which stands for Cascading Style Sheets.

II. Validation and error messages

A well-formed HTML page should strictly follow the rules defined by the HTML specification. Adhering to these rules ensures consistency, compatibility, and proper rendering across different Web browsers and devices. When your HTML code is well-formed, it means that it follows the syntax, structure, and guidelines outlined in the HTML specification. Most of the browsers sometimes ignore the simple syntax mistakes but that can also cause some undetected errors.

Validating your HTML code and analyzing error messages is crucial for ensuring your Web pages are well-formed and compliant with HTML standards. Here's how you can perform validation and interpret error messages in an HTML5 page:

1. Performing Validation:

Using Online Validators:

There are online tools provided by the W3C that allow you to validate your HTML code. One such tool is the W3C Markup Validation Service (<https://validator.w3.org/>).

- Visit the W3C Markup Validation Service Website.
- Paste your HTML code into the input field.
- Click the "Check" or "Validate" button.
- The validator will analyze your code and provide feedback.

Browser Developer Tools:

Most modern Web browsers have built-in developer tools that can help you identify HTML errors.

- Right-click on your Web page and select "Inspect" or use the shortcut key (usually F12 or Ctrl+Shift+I) to open the developer tools.
- Navigate to the "Console" tab. HTML errors will be displayed here along with error messages.

2. Analyzing Error Messages:

Understanding Error Types:

Syntax Errors: Syntax errors occur when the HTML code does not follow the correct syntax rules. These occur when there are mistakes in the structure of your HTML code, such as missing closing tags or incorrectly nested elements. The error message will specify the location of the error. These errors can lead to issues with rendering or executing the HTML.

Example: Missing Closing Tag

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
</head>
<body>
  <h1>Hello, World!
</body>
</html>
```

In this example, the closing tag for the <h1> element is missing, causing a syntax error.

Syntax Errors Checking:

Sometimes browsers autocorrect some of the errors. For this reason, even a malformed HTML page will show no error while opening with internet browsers. But we can easily find the error through the online validation service from <https://validator.w3.org/>.

Let us create an HTML page with deliberate error:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Error Page</title>
</head>
```

```
<body>

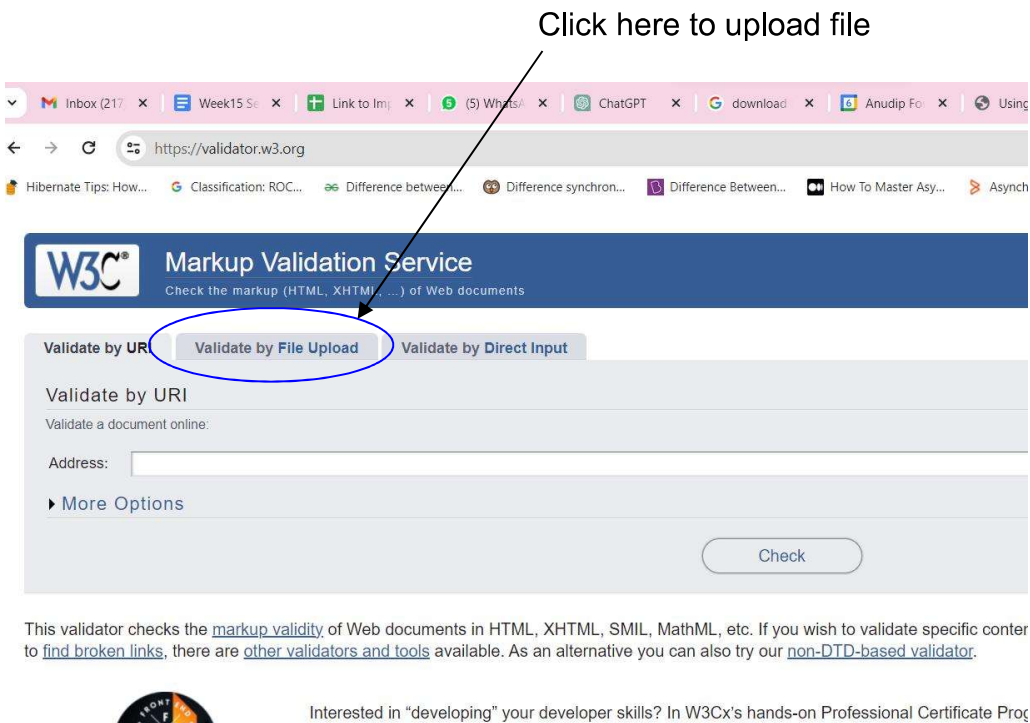
  <h1>This is a Page with Syntax Error

  <p>This paragraph has no closing tag.

  <wrongTag> A wrong tag</wrongTag>
</body>
</html>
```

In this example we have deliberately introduced a few errors like not closing tags and using wrong tags. Let us save this HTML file, visit the <https://validator.w3.org/> and upload this HTML file.

Click here to upload file



W3C® Markup Validation Service
Check the markup (HTML, XHTML, ...) of Web documents

Validate by URI Validate by **File Upload** Validate by Direct Input

Validate by URI
Validate a document online:

Address:

► More Options

Check

This validator checks the [markup validity](#) of Web documents in HTML, XHTML, SMIL, MathML, etc. If you wish to validate specific content to [find broken links](#), there are [other validators and tools](#) available. As an alternative you can also try our [non-DTD-based validator](#).

Interested in "developing" your developer skills? In W3C's hands-on Professional Certificate Program

After uploading the HTML file click on the "Check" button.

You will see that the page errors are listed :

- Error** Element `p` not allowed as child of element `h1` in this context. (Suppressing further errors from this subtree.)
 From line 12, column 5; to line 12, column 7

```

      <h1>
      <p>This p
      
```

 Contexts in which element `p` may be used:
 Where `flow content` is expected.
- Error** End tag for `body` seen, but there were unclosed elements.
 From line 15, column 1; to line 15, column 7

```

      <body>
      <wrongTag>
      </body>
      </html>
      
```
- Error** Unclosed element `h1`.
 From line 10, column 5; to line 10, column 8

```

      <body>
      <h1>This i
      
```

Attribute Errors: These occur when HTML attributes are used incorrectly or have invalid values.

Example: Incorrect Attribute Name

```

<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    
  </body>
</html>

```

In this example, the attribute “hrwidth” is used incorrectly for the `<img>` element instead of width, causing an attribute error.

Attribute Errors Checking:

Now let us verify attribute errors in HTML Page. Let us save the above HTML file, and verify the error using validator at <https://validator.w3.org/>. The error message will be:

1. **Error** Attribute `hrwidth` not allowed on element `img` at this point.

From line 11, column 5; to line 11, column 57

```
</bod
```

Attributes for element `img`:

[Global attributes](#)

[alt](#) — Replacement text for use when images are not available

[src](#) — Address of the resource

[srcset](#) — Images to use in different situations, e.g., high-resolution displays, small monitors, etc.

[sizes](#) — Image sizes for different page layouts

[crossorigin](#) — How the element handles crossorigin requests

[usemap](#) — Name of [image map](#) to use

[ismap](#) — Whether the image is a server-side image map

[width](#) — Horizontal dimension

[height](#) — Vertical dimension

[referrerpolicy](#) — [Referrer policy](#) for [fetches](#) initiated by the element

[decoding](#) — Decoding hint to use when processing this image for presentation

[loading](#) — Used when determining loading deferral

[fetchpriority](#) — Sets the [priority](#) for [fetches](#) initiated by the element

Element Errors: These occur when elements are used incorrectly or are nested improperly, violating HTML5 rules.

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
</head>
<body>
  <p>This is a <em>paragraph with <strong>strong emphasis</em>.</strong></p>
</body>
</html>
```

In this example, the `<em>` element is not properly closed before the `<strong>` element, causing a nesting error.

Element Errors Checking:

Let us save the above HTML file, and verify the error using validator at <https://validator.w3.org/>. The error message will be:



Warnings: Sometimes, validators might issue warnings about best practices, accessibility, or deprecated features. While these are not errors, it's good to address them for better code quality.

Locating Errors:

Error messages usually provide information about the line number and specific issue in your HTML code. Look for this information to identify the problematic area.

Pay attention to the context of the error. Understanding the surrounding code can often provide clues about the cause of the issue.

Use indentation and proper formatting to make it easier to spot nested elements and their relationships.

Fixing Errors:

Once you identify the error, go back to your HTML code and find the specified line.

Review the code around the error to understand the context and fix the issue. Common problems include missing closing tags, mismatched tags, and incorrect attribute usage.

By validating your HTML code and understanding error messages, you can ensure your HTML5 pages are error-free and compliant with Web standards, leading to better user experiences and improved Website performance.

III. Audio and Video in HTML5

In HTML5, you can easily embed audio and video files into Web pages using the `<audio>` and `<video>` elements, respectively. Here's how you can do it:

Inserting Audio:

To insert audio, you can use the `<audio>` element. You need to specify the source of the audio file using the `src` attribute. You can also provide fallback content between the opening and closing `<audio>` tags, which will be displayed if the browser doesn't support the `<audio>` element.

Example of embedding audio:

```
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

In this example, replace "audio.mp3" with the actual file path of your audio file. The controls attribute adds play, pause, and volume controls to the audio player.

Inserting Video:

To insert video, you can use the <video> element. Similar to <audio>, you specify the video file source using the src attribute. You can also include multiple <source> elements inside the <video> element to provide different formats of the video for better browser compatibility. The browser will automatically select the first supported format.

Example of embedding video:

```
<video controls>
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Your browser does not support the video element.
</video>
```

In this example, replace "video.mp4" and "video.webm" with the actual file paths of your video files. The controls attribute adds play, pause, and volume controls to the video player.

Optional Attributes:

- **controls:** Adds playback controls (play, pause, volume) to the audio or video player.
- **autoplay:** Starts playing the media automatically when the page loads.
- **loop:** Causes the media to replay from the beginning after it reaches the end.
- **muted:** Starts the media with the sound turned off.
- **width and height:** Specifies the width and height of the video element.

Remember to provide alternative text or content between the opening and closing tags of the <audio> and <video> elements. This content will be displayed in browsers that do not support these elements or if the media file cannot be loaded.

Additionally, always provide multiple formats of audio and video files to ensure compatibility across different browsers. Common formats include MP3, OGG, and WAV for audio, and MP4, WebM.

Here is an example of HTML 5 page that includes both audio and video tag HTML5. Please replace the name of the audio and video file with the actual files you have in

your local file system. The files should reside in the same directory where the HTML file is located.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Audio and Video Example</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      margin: 0;
      padding: 20px;
      display: flex;
      flex-direction: column;
      align-items: center;
      text-align: center;
    }
    header {
      color: #280850;
      text-align: center;
      padding: 10px;
    }

    h1 {
      color: #333;
      margin-bottom: 20px;
    }

    p {
      color: #123;
      margin-bottom: 10px;
      font-size: 20px;
      text-align: left;
      font-weight: 500;
    }

    table {
      width: 100%;
      margin-top: 20px;
      border-collapse: collapse;
```

```

    }

    th, td {
        border: 1px solid #ddd;
        padding: 10px;
        text-align: center;
    }

    th {
        background-color: #3498db;
        color: #fff;
    }

    audio, video {
        width: 100%;
        max-width: 640px;
    }
</style>
</head>

<body>
<header> <h1>Audio and Video Example</h1> </header>

<table>
  <tr>
    <th>Audio Example</th>
  </tr>
  <tr>
    <td>
      <audio controls>
        <source src="audio.mp3" type="audio/mpeg">
        Your browser does not support the audio element.
      </audio>
    </td>
  </tr>
</table>

<table>
  <tr>
    <th>Video Example</th>
  </tr>
  <tr>
    <td>
      <video controls>
        <source src="video.mp4" type="video/mp4">
        <source src="video.webm" type="video/webm">

```

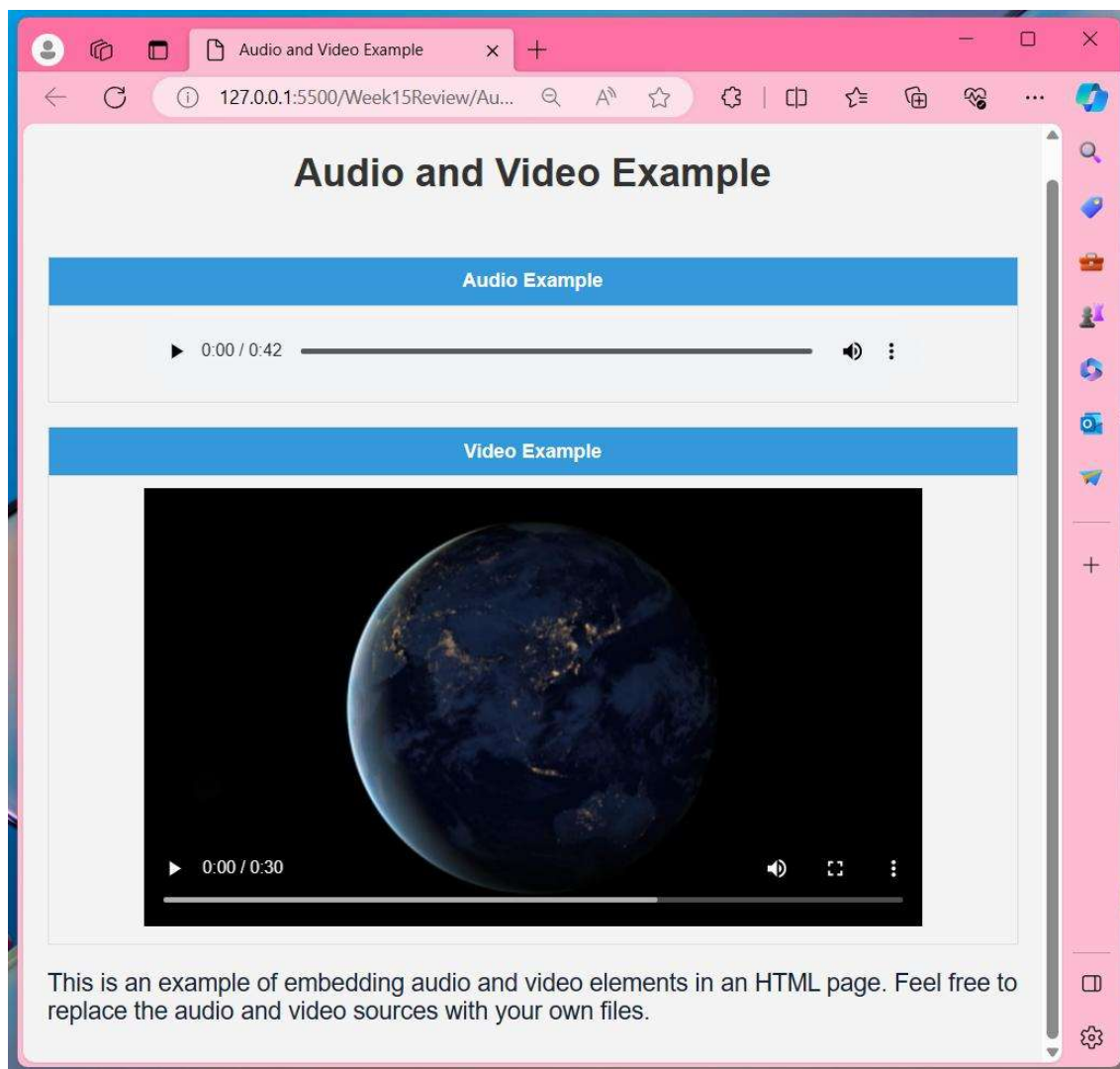
```
        Your browser does not support the video element.  
    </video>  
  </td>  
</tr>  
</table>
```

<p>This is an example of embedding audio and video elements in an HTML page. Feel free to replace the audio and video sources with your own files.</p>

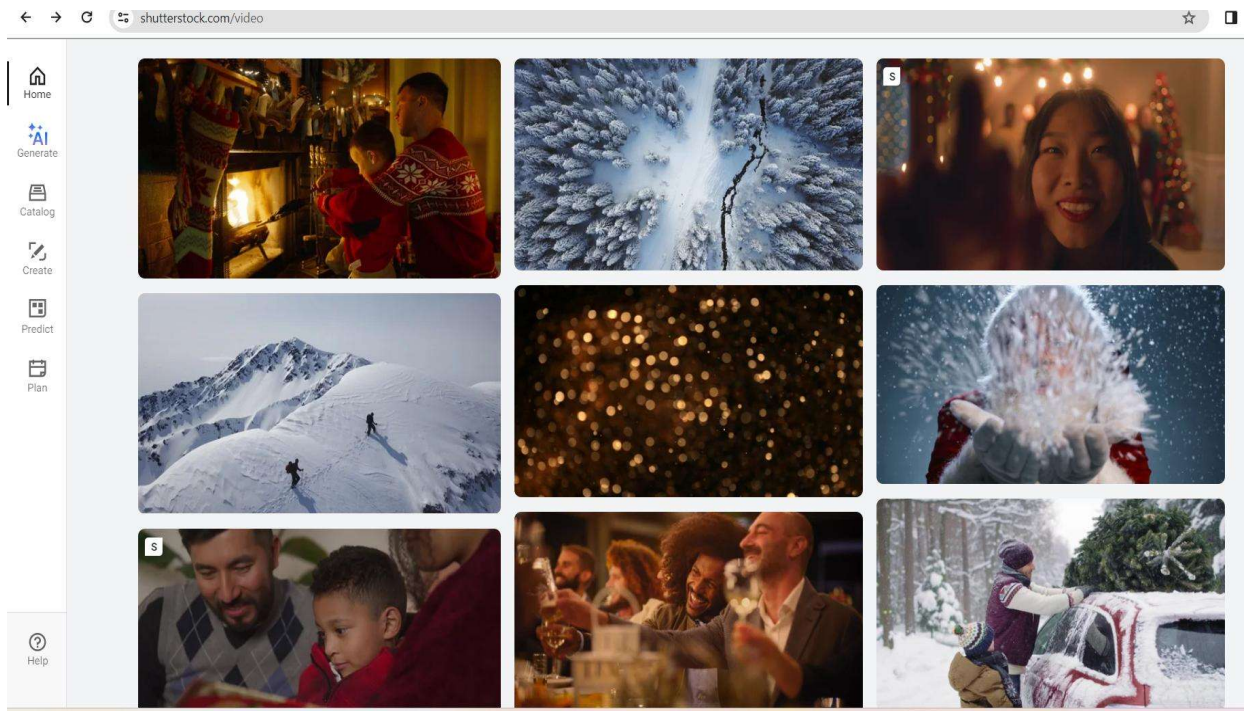
</body>

</html>

If you open the file in browser it will look like this:



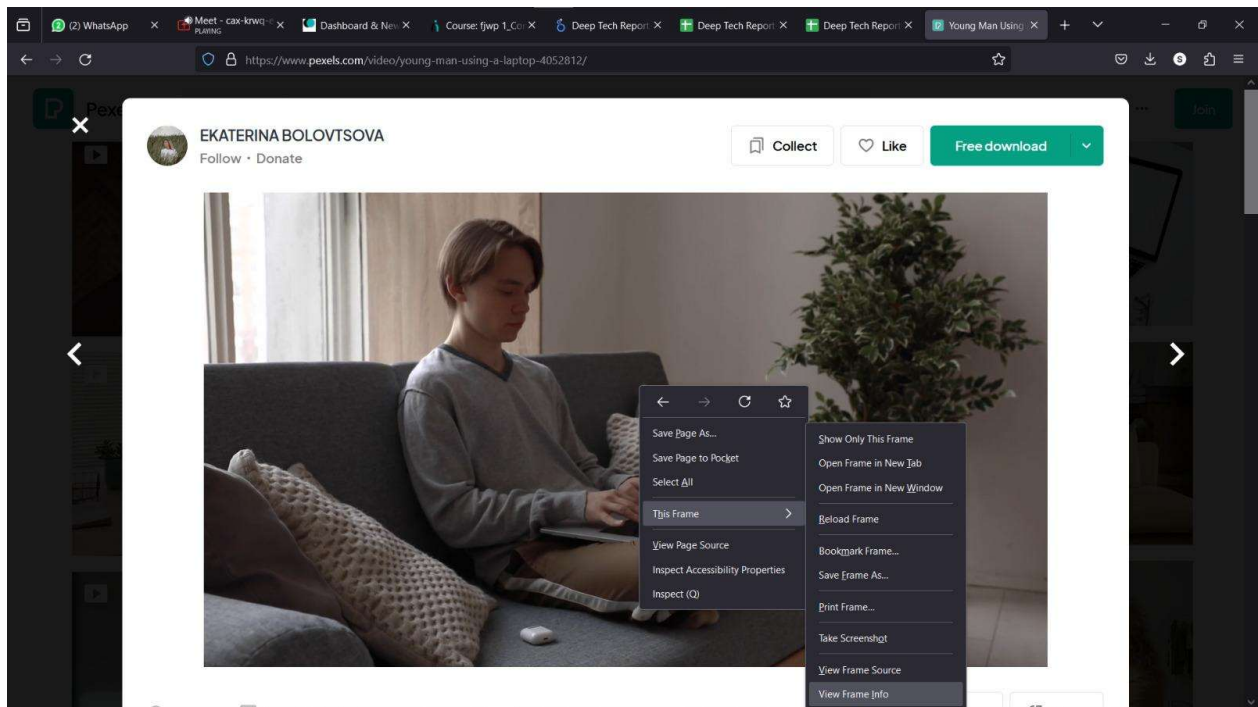
Let us take a scenario of a shutterstock <https://www.shutterstock.com/> video gallery where we can play the audio with video. The website look like as below:



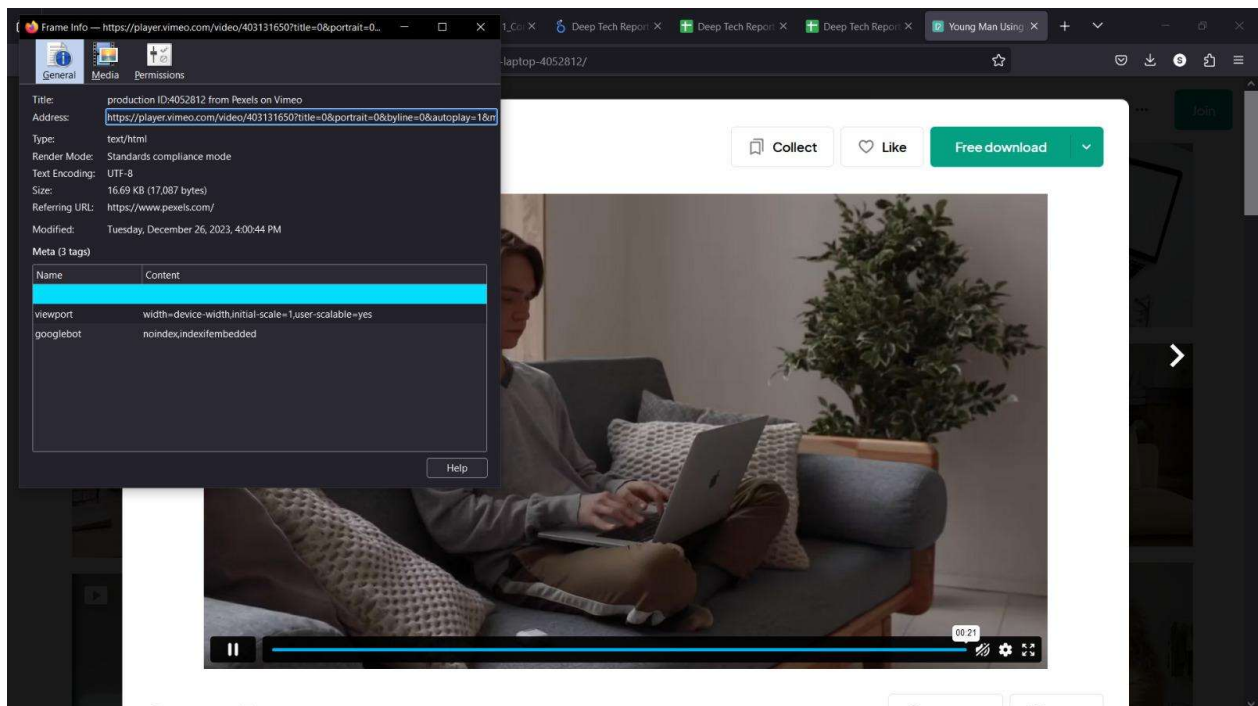
To create a video gallery we will copy the url of some video files from the browser.

You can select any video url from any website. Let us see how to copy the url and add it into our html application.

Open a browser and go to www.pexels.com. Right click on any video and go to This frame and select the view frame info.



Copy the address of the url.



Let us create a video gallery using HTML5 audio & video files.

```
<!DOCTYPE html>
<html lang="en">
```

```

<head>
  <meta charset="UTF-8">
  <title>Video Paradise</title>
  <link href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body{
      background-color: aliceblue;
    }
    .embed-responsive
    {
      border-radius: 10px;
      overflow: hidden;
    }

  </style>
</head>

<body>

  <div class="container mt-3">
    <h1 class="text-center">Video Paradise</h1>
  </div>

  <div class="container mt-5">
    <div class="row">
      <div class="col-md-4 mb-4">
        <!--Create responsive video or slideshow embeds based on the width of the parent by
        creating an intrinsic ratio that scales on any device -->
        <div class="embed-responsive embed-responsive-16by9">

          <!--embed specific content from one web page — like a video, form, document, or even
          a full web page-->
          <iframe class="embed-responsive-item"
src="https://player.vimeo.com/video/887957612?title=0&portrait=0&byline=0&autoplay=
1&muted=true"
allowfullscreen></iframe>
        </div>
      </div>

      <div class="col-md-4 mb-4">
        <div class="embed-responsive embed-responsive-16by9">
          <iframe class="embed-responsive-item"

```

src="https://player.vimeo.com/video/885682206?title=0&portrait=0&byline=0&autoplay=1&muted=true"

allowfullscreen></iframe>

</div>

</div>

<div class="col-md-4 mb-4">

<div class="embed-responsive embed-responsive-16by9">

<iframe class="embed-responsive-item"

src="https://player.vimeo.com/video/885752744?title=0&portrait=0&byline=0&autoplay=1&muted=true"

allowfullscreen></iframe>

</div>

</div>

</div>

<div class="row">

<div class="col-md-4 mb-4">

<div class="embed-responsive embed-responsive-16by9">

<iframe class="embed-responsive-item"

src="https://player.vimeo.com/video/520529890?title=0&portrait=0&byline=0&autoplay=1&muted=true"

allowfullscreen></iframe>

</div>

</div>

<div class="col-md-4 mb-4">

<div class="embed-responsive embed-responsive-16by9">

<iframe class="embed-responsive-item"

src="https://player.vimeo.com/video/880567165?title=0&portrait=0&byline=0&autoplay=1&muted=true"

allowfullscreen></iframe>

</div>

</div>

<div class="col-md-4 mb-4">

<div class="embed-responsive embed-responsive-16by9">

<iframe class="embed-responsive-item"

src="https://player.vimeo.com/video/887041557?title=0&portrait=0&byline=0&autoplay=1&muted=true"

allowfullscreen></iframe>


```

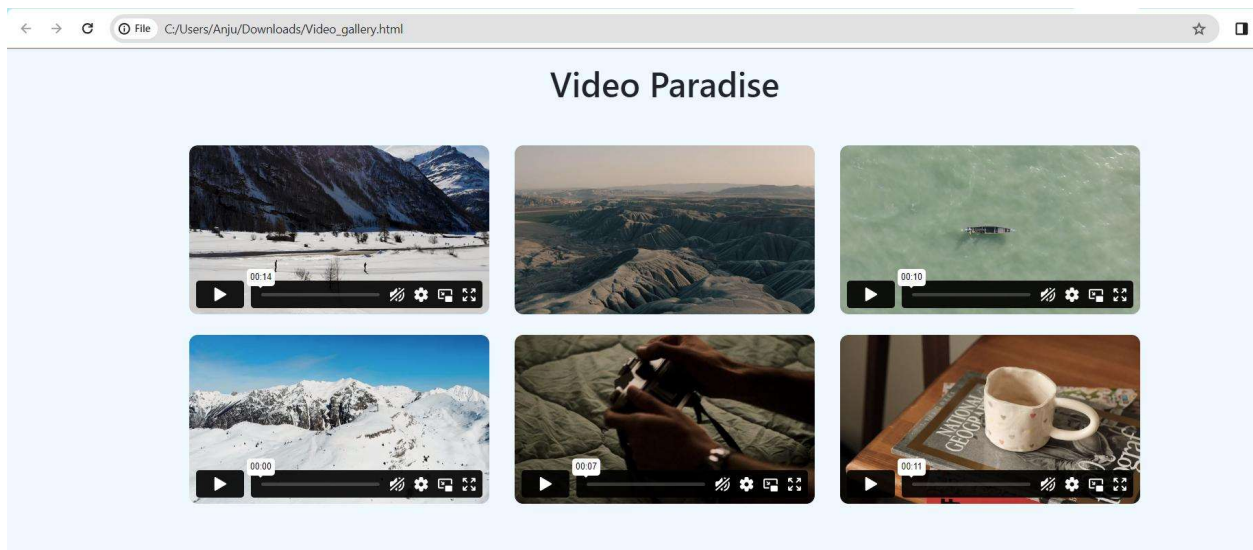
        </div>
    </div>
</div>
</div>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.16.0/umd/popper.min.js"></script>
>
<script
src="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></script>
</body>

</html>

```

Output:



IV. Adding multimedia content with HTML5

Other than audio and video, you can add several other multimedia content to your HTML5 Web pages using various elements and attributes. Here are the primary ways to include other multimedia content:

1. Canvas:

The <canvas> element in HTML5 is a useful feature that provides a drawing surface for graphics using JavaScript. It allows you to dynamically create and manipulate graphics, charts, animations, and other visual elements directly within the browser. In this session we will only give a brief overview of Canvas. Detailed explanation is not possible at this stage because JavaScript is not taught so far. All the canvas features will be explained with examples after JavaScript is taught.

Example:


```
<canvas id="myCanvas" width="200"
      height="100" style="border:1px solid black;">
</canvas>
```

Here is an example of using canvas in HTML5.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Canvas Example</title>
</head>
<body>

  <h1>Canvas Example</h1>

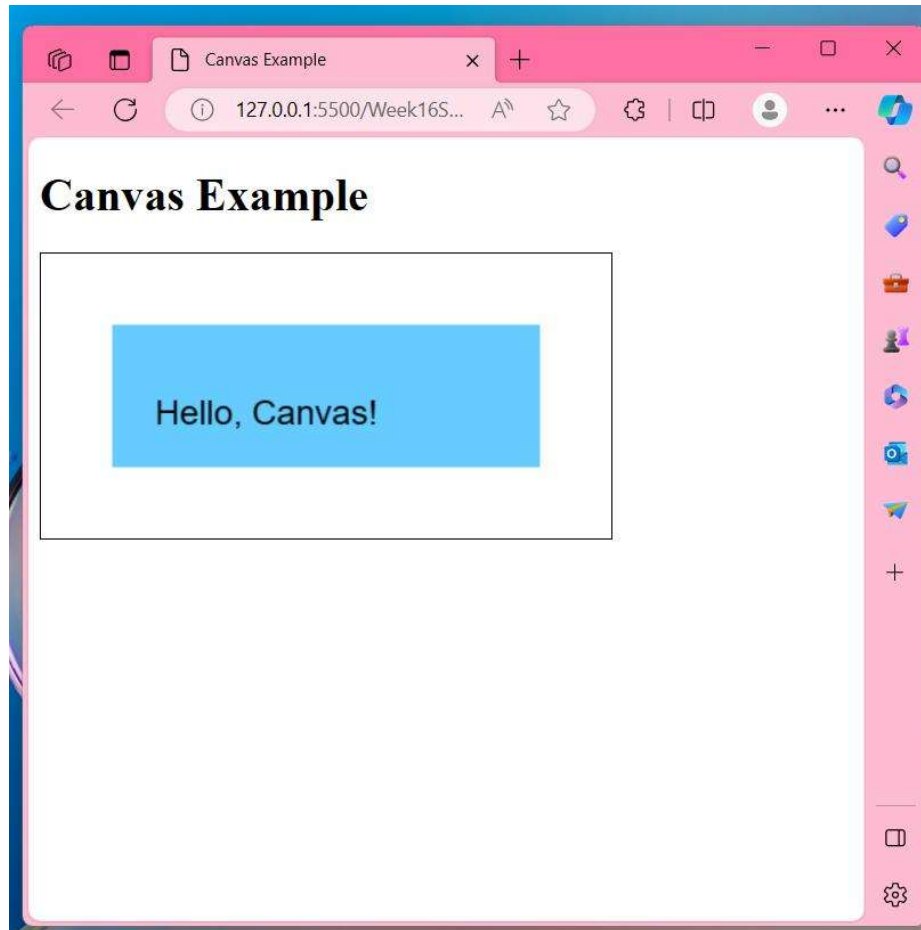
  <!-- Canvas element with a specified width and height -->
  <canvas id="myCanvas" width="400" height="200" style="border:1px solid
#000;"></canvas>

  <script>
    // Get the canvas element and its drawing context
    var canvas = document.getElementById('myCanvas');
    var ctx = canvas.getContext('2d');

    // Draw a rectangle on the canvas
    ctx.fillStyle = '#66ccff'; // Set fill color to light blue
    ctx.fillRect(50, 50, 300, 100); // Draw a filled rectangle

    // Draw text on the canvas
    ctx.fillStyle = '#000'; // Set text color to black
    ctx.font = '24px Arial';
    ctx.fillText('Hello, Canvas!', 80, 120);
  </script>
</body>
</html>
```

Page will look like this in the browser.



Canvas will be covered in detail in week 16 after studying JavaScript.

2. SVG (Scalable Vector Graphics):

SVG (Scalable Vector Graphics) is an XML-based vector image format used for describing two-dimensional vector graphics. Unlike raster graphics, which are made up of pixels, SVG graphics are scalable and resolution-independent, making them ideal for various applications such as Web design, data visualization, and interactive graphics. SVG supports shapes, paths, text, and gradients, allowing for the creation of detailed and dynamic graphics.

So in a nutshell:

- SVG stands for Scalable Vector Graphics
- SVG is used to define graphics for the Web
- SVG is a W3C recommendation

We use **<svg>** tag to draw SVG, <svg> element is a container for SVG graphics. That means all SVG graphics should be drawn within <svg> and </svg> tags.

SVG has several methods for drawing paths, boxes, circles, text, and graphic images.

Key Features of SVG:

Scalability: SVG images can be scaled to any size without losing quality, making them ideal for responsive Web design.

Interactivity: SVG supports interactivity through JavaScript, allowing developers to create dynamic and engaging graphics.

Accessibility: SVG content can be easily styled and manipulated using CSS, making it accessible and customizable.

Text-Based: SVG files are human-readable and can be created and edited with a simple text editor.

Animation: SVG supports animation through SMIL (Synchronized Multimedia Integration Language) or JavaScript, enabling the creation of dynamic visual effects.

Examples of SVG:

SVG has many useful features to draw graphics components. Here are a few simple examples to illustrate the use of SVG:

Basic Shapes:

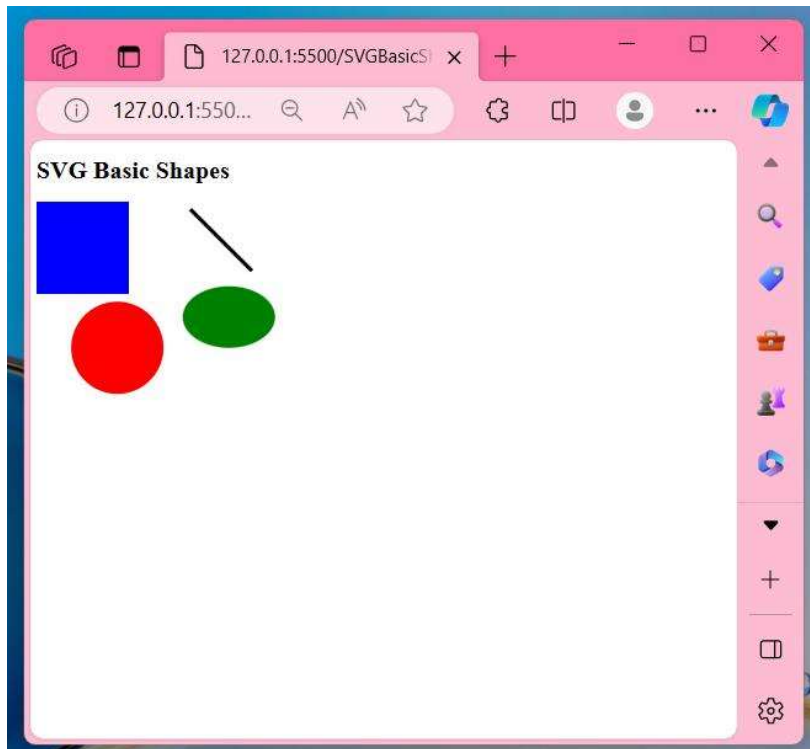
We can create basic shapes like Rectangle, Circle and Eclipse using SVG.

```
<!DOCTYPE html>
<html>
<body>
  <h1> SVG Basic Shapes</h1>

  <svg width="450" height="450" xmlns="http://www.w3.org/2000/svg">
    <rect width="120" height="120" fill="blue" />
    <circle cx="105" cy="190" r="60" fill="red" />
    <ellipse cx="250" cy="150" rx="60" ry="40" fill="green" />
    <line x1="200" y1="10" x2="280" y2="90" stroke="black" stroke-width="5" />
  </svg>

</body>
</html>
```

If you open the HTML file in a browser, you can see the following shapes:



Path Element:

The SVG `<path>` element is a fundamental and versatile element that allows you to define complex shapes and paths. It is commonly used to create shapes, curves, and lines by specifying a series of commands within the `d` attribute. The `d` attribute contains a set of path data that instructs the browser on how to draw the path.

Here's an explanation of some basic commands used in the `d` attribute:

M (move to): Moves the pen to a specified point without drawing.

L (line to): Draws a straight line from the current point to a specified point.

H (horizontal line to): Draws a horizontal line to a specified x-coordinate.

V (vertical line to): Draws a vertical line to the specified y-coordinate.

C (cubic Bezier curve): Draws a cubic Bezier curve with control points.

S (smooth cubic Bezier curve): Draws a cubic Bezier curve with a single control point.

Q (quadratic Bezier curve): Draws a quadratic Bezier curve with control points.

T (smooth quadratic Bezier curve): Draws a quadratic Bezier curve with a single control point.

A (elliptical arc): Draws an elliptical arc.

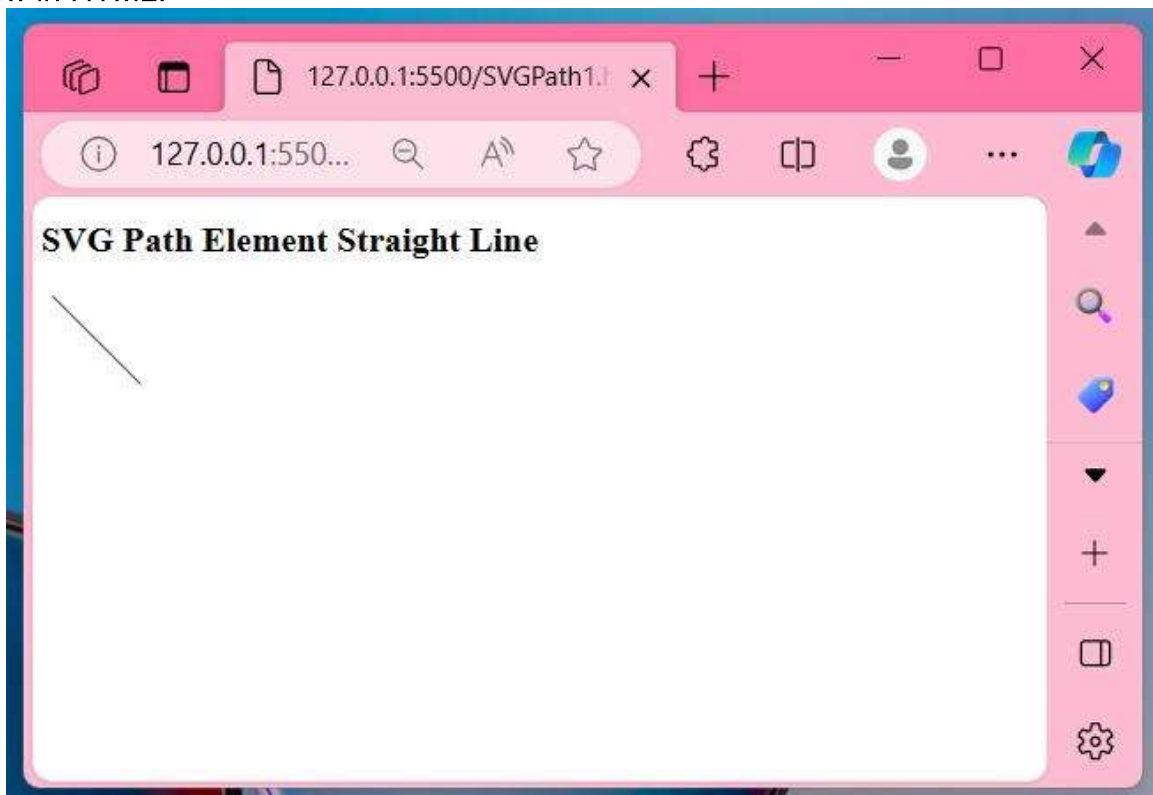
Z (close path): Closes the current path by connecting the current point to the starting point.

Here are a few examples to illustrate the use of the <path> element:

Straight Line:

```
<!DOCTYPE html>
<html>
<body>
<h1> SVG Path Element Straight Line</h1>
  <svg width="100" height="100" xmlns="http://www.w3.org/2000/svg">
    <path d="M10 10 L90 90" stroke="black" fill="transparent" />
  </svg>
</body>
</html>
```

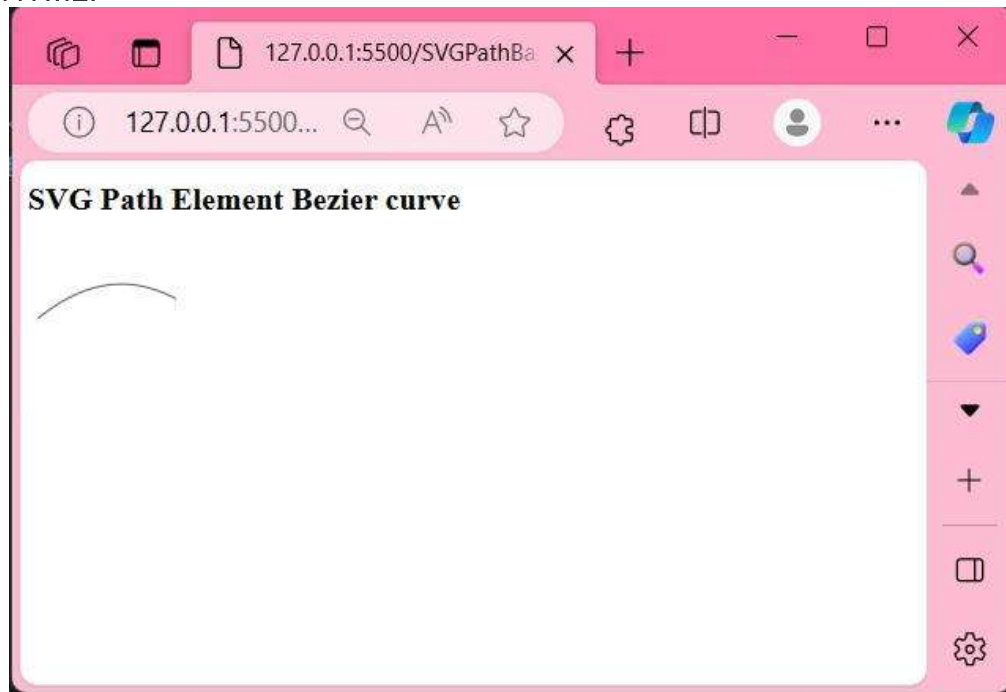
View in HTML:



Quadratic Bezier Curve:

```
<svg width="150" height="100" xmlns="http://www.w3.org/2000/svg">
  <path d="M10 80 Q 95 10 180 80" stroke="black" fill="transparent"/>
</svg>
```

View in HTML:



Cubic Bezier Curve:

```
<svg width="150" height="100" xmlns="http://www.w3.org/2000/svg">  
  <path d="M10 80 C 40 10, 65 10, 95 80 S 150 150, 180 80" stroke="black"  
  fill="transparent"/>  
</svg>
```

Elliptical Arc:

```
<svg width="150" height="100" xmlns="http://www.w3.org/2000/svg">  
  <path d="M10 80 A 45 45, 0, 0, 1, 180 80" stroke="black" fill="transparent"/>  
</svg>
```

These examples demonstrate how to use the `<path>` element to create different shapes and curves. Understanding the commands and their parameters allows you to draw a wide range of paths and shapes, making the `<path>` element a powerful tool for creating complex vector graphics in SVG.

Gradient:

SVG gradients are a way to fill shapes or backgrounds in Scalable Vector Graphics (SVG) with smoothly transitioning colors. Gradients allow for the creation of smooth color transitions, providing depth and realism to vector graphics. There are two main types of gradients in SVG: linear gradients and radial gradients.

1. Linear Gradients:

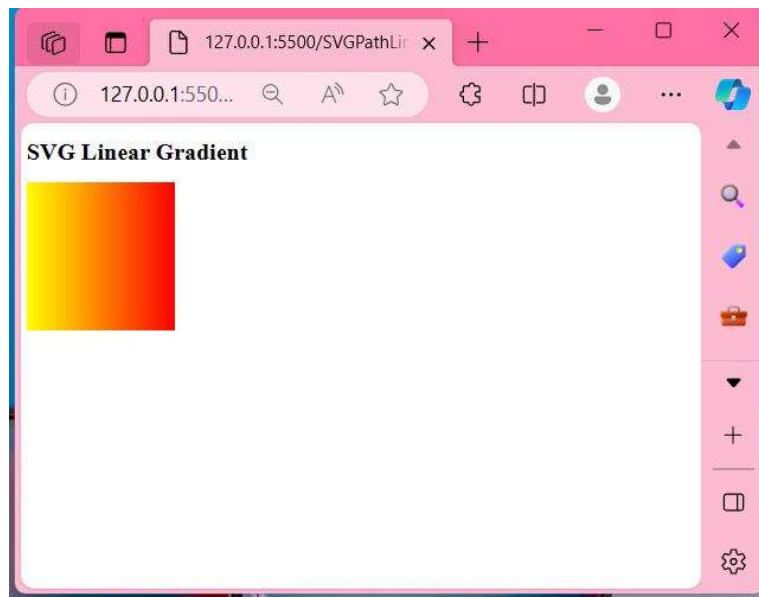
Linear gradients create a transition between colors along a straight line. The line is defined by an imaginary axis, and the colors smoothly blend from one point to another.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h1> SVG Linear Gradient</h1>
<svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
  <defs>
    <linearGradient id="grad1" x1="0%" y1="0%" x2="100%" y2="0%">
      <stop offset="0%" style="stop-color:rgb(255,255,0);stop-opacity:1" />
      <stop offset="100%" style="stop-color:rgb(255,0,0);stop-opacity:1" />
    </linearGradient>
  </defs>
  <rect width="200" height="200" fill="url(#grad1)" />
</svg>
</body>
</html>
```

In this example, a linear gradient is applied to a rectangle. The gradient goes from yellow to red horizontally.

View in HTML:



2. Radial Gradients:

Radial gradients create a transition between colors in a circular pattern, radiating outward from a central point.

Example:

```
<svg width="400" height="400" xmlns="http://www.w3.org/2000/svg">
```

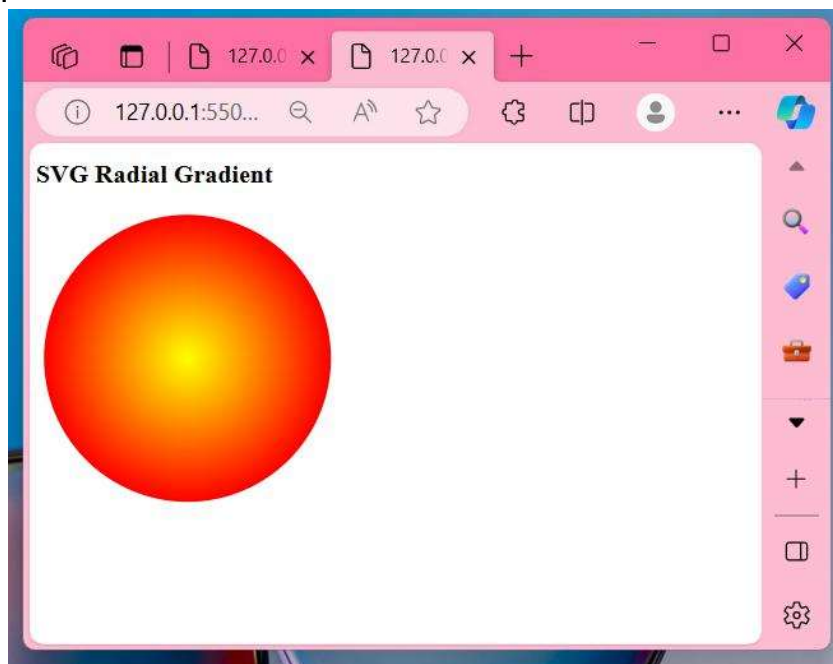
```

<defs>
  <radialGradient id="grad2" cx="50%" cy="50%" r="50%" fx="50%"
  fy="50%">
    <stop offset="0%" style="stop-color:rgb(255,255,0);stop-opacity:1" />
    <stop offset="100%" style="stop-color:rgb(255,0,0);stop-opacity:1" />
  </radialGradient>
</defs>
<circle cx="190" cy="100" r="180" fill="url(#grad2)" />
</svg>

```

In this example, a radial gradient is applied to a circle. The gradient goes from yellow to red radially.

View in HTML:



Gradient Attributes:

- **id:** A unique identifier for the gradient, allowing it to be referenced in the fill or stroke attributes.
- **x1, y1, x2, y2 (Linear Gradient):** Defines the start and end points of the gradient line.
- **cx, cy, r, fx, fy (Radial Gradient):** Define the position and size of the radial gradient.

Multiple Stops:

Gradients can have multiple color stops, creating more complex color transitions. Each stop is defined with an offset attribute indicating the position along the gradient line or circle.

Example with Multiple Stops:

```
<!DOCTYPE html>
```



```

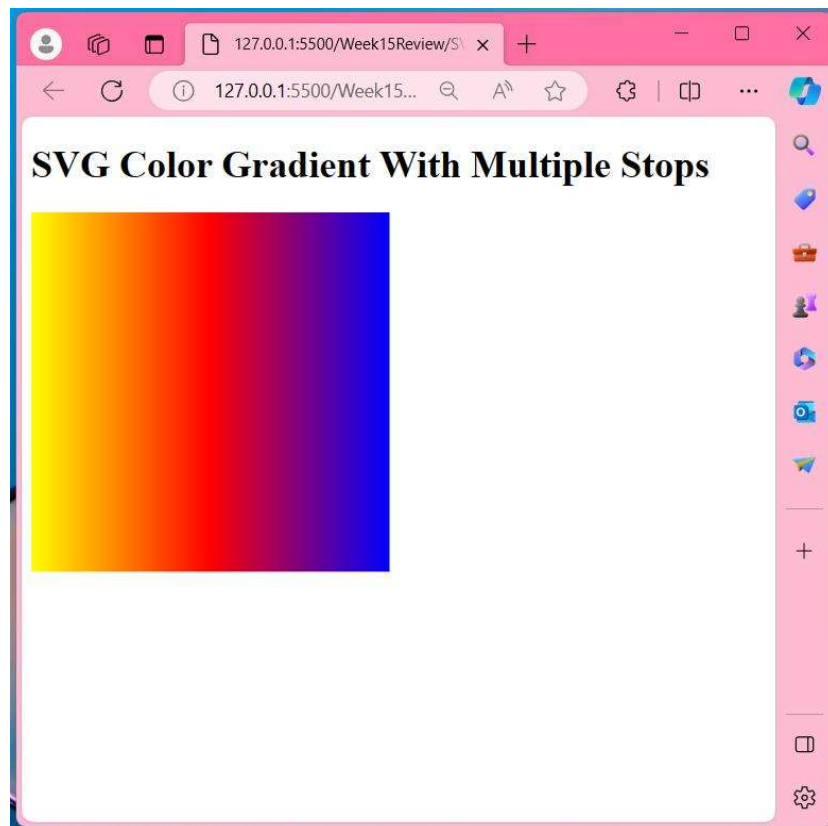
<html>
<body>
<h1> SVG Color Gradient With Multiple Stops</h1>
<svg width="400" height="400" xmlns="http://www.w3.org/2000/svg">

  <defs>
    <linearGradient id="grad1" x1="0%" y1="0%" x2="100%" y2="0%">
      <stop offset="0%" style="stop-color:rgb(255,255,0);stop-opacity:1" />
      <stop offset="50%" style="stop-color:rgb(255,0,0);stop-opacity:1" />
      <stop offset="100%" style="stop-color:rgb(0,0,255);stop-opacity:1" />
    </linearGradient>
  </defs>
  <rect width="300" height="300" fill="url(#grad1)" />
</svg>

</body>
</html>

```

In this example, a linear gradient transitions from yellow to red to blue.



SVG gradients are powerful tools for enhancing the visual appeal of vector graphics, providing a way to smoothly transition between colors and create more complex and realistic visual effects.

These examples demonstrate the versatility of SVG, allowing you to create a variety of shapes, paths, text, and gradients. As you delve deeper into SVG, you can explore its animation capabilities, filters, and other advanced features to create rich and interactive graphics.

3. Embedded Content:

Use the <iframe> element to embed content from other Websites, such as videos from YouTube or maps from Google Maps.

Example:

```
<iframe src="https://www.youtube.com/embed/examplevideo" width="560" height="315" frameborder="0" allowfullscreen></iframe>
```

Remember to provide alternative text or fallback content for multimedia elements, especially for images, audio, and video, by using the alt attribute or text content between the opening and closing tags. This ensures accessibility and provides a meaningful experience for users, including those with disabilities or users on slow internet connections.

V. Introduction to Cascading Style Sheet (CSS)

In creating a responsive Web based user interface we use a standard tool called CSS (Cascading Style Sheets) along with HTML. The literal meaning of Cascading is a waterfall or series of small waterfalls. The term cascading is used to describe flow.

In the term CSS, Cascading means that styles are applied in a hierarchical manner, with the most specific styles taking precedence over more general ones. Style sheets are written in a text-based format that is easy to learn and use.

VI. Why CSS is needed in a Web site

CSS (Cascading Style Sheets) is essential in Web development for several reasons:

1. Separation of Concerns:

CSS allows the separation of content (HTML) from presentation (styling). This separation makes the codebase more organized, easier to maintain, and more flexible. Design changes can be made without altering the underlying HTML structure.

2. Consistent Styling:

CSS enables consistent styling across a Website. By defining styles in a central CSS file, you can apply the same look and feel to multiple pages, ensuring a cohesive user experience.

3. Responsive Web Design:

CSS allows you to create responsive layouts that adapt to different screen sizes and devices. With techniques like media queries, you can apply specific styles for various screen widths, making your Website accessible and usable on desktops, tablets, and smartphones.

4. Improved User Experience:

Well-designed CSS can enhance the user experience by improving readability, accessibility, and overall aesthetics. Proper use of typography, colors, spacing, and other visual elements can make content more engaging and user-friendly.

5. Faster Loading Times:

CSS files can be cached by browsers, reducing the need to download stylesheets repeatedly. This caching mechanism helps in faster loading times, improving the overall performance of your Website.

6. SEO (Search Engine Optimization):

CSS can indirectly impact SEO by enhancing the user experience. Search engines tend to favor Websites that provide a positive user experience, and well-designed CSS can contribute to this by improving readability and accessibility.

7. Easy Maintenance:

By centralizing styles in CSS files, it's easier to update the appearance of a Website. You can make global changes by modifying a few lines of code in the CSS file, rather than editing each individual HTML file.

8. Print-Friendly Layouts:

CSS allows you to create print styles, ensuring that your Web pages are presented in a printer-friendly format. This is particularly important for Websites that provide content meant for printing, such as articles or reports.

9. Animations and Interactivity:

CSS can be used to create animations and interactive elements without the need for JavaScript. This includes transitions, transformations, and keyframe animations, enhancing the visual appeal and user engagement of a Website.

In summary, CSS is crucial for creating visually appealing, responsive, and user-friendly Websites. It not only improves the presentation of content but also plays a significant role in optimizing performance and enhancing the overall user experience.

VII. Overview of CSS

CSS, is a very useful technology used to control the visual presentation of Web pages. It allows Web developers to define the layout, styling, and formatting of HTML elements. CSS is a separate language from HTML, but it is used in conjunction with HTML to create visually appealing and consistent Web designs.

Here's an overview of CSS in HTML:

Separation of Content and Presentation: One of the key principles of Web development is the separation of content (HTML) and presentation (CSS). HTML is responsible for structuring the content of a Web page, while CSS is responsible for defining how that content is styled and displayed.

Selectors and Properties: CSS uses selectors to target HTML elements and apply styling rules to them. Selectors can be based on element types, classes, IDs, or other attributes. Properties are used to specify the visual characteristics of the selected elements, such as color, font, size, margin, padding, and more.

Style Rules: CSS rules are defined as a combination of selectors and properties. A simple CSS rule might look like this:

```
p {  
    font-size: 16px;  
    color: #333;  
}
```

In this example, the selector `p` targets all paragraph elements, and the properties `font-size` and `color` specify the styling for those paragraphs.

Inline, Internal, and External CSS: You can include CSS in HTML documents in three different ways:

Inline CSS: You can apply styles directly within the HTML tags using the `style` attribute. For example:

```
<p style="color: blue; font-size: 18px;">This is a blue paragraph.</p>
```

Internal CSS: You can include CSS rules within the HTML `<style>` element in the document's `<head>` section. We have already used the `<style>` tag in the earlier sessions.

For example:

```
<head>  
  <style>  
    p {  
      color: blue;  
      font-size: 18px;  
    }  
  </style>  
</head>
```

External CSS: You can create a separate CSS file with a .css extension and link it to your HTML document using the <link> element in the <head> section. For example:

For eg.

Consider the below styles.css file

```
body {  
    background-color: lightblue;  
}  
  
h1 {  
    color: white;  
    text-align: center;  
}  
  
p {  
    font-family: verdana;  
    font-size: 20px;  
}
```

We can use this external file in any HTML page as follows.

```
<head>  
    <link rel="stylesheet" type="text/css" href="styles.css">  
</head>
```

Note: We will discuss external style sheets in detail in the upcoming session.

Cascading and Specificity: CSS follows a set of rules to determine the final styles applied to an element when multiple conflicting styles are defined. These rules involve the concept of specificity, inheritance, and the cascade. More specific selectors or styles closer to the element being styled take precedence.

CSS is an essential part of Web development that allows you to control the visual appearance of your HTML content. It enables you to create attractive, consistent, and responsive Web designs while maintaining a clean separation of content and presentation.

VIII. CSS syntax and selectors

CSS syntax is relatively simple and easy to learn. A CSS rule consists of a selector, a property, and a value. The selector specifies which HTML element the rule should apply to, the property specifies the style to be applied, and the value specifies the value of the property.

For example, the following CSS rule will set the text color of all paragraph elements to red:

```
p {  
  color: red;  
}
```

CSS Selectors

CSS selectors are used to identify the HTML elements that a CSS rule should apply to. There are three main types of CSS selectors:

- **Element selectors:** These selectors select all elements of a specific type. For example, the selector `p` selects all paragraph elements.

Element selectors in CSS target specific HTML elements for styling. They are the most basic type of selector and are defined by the name of the HTML element. Here are some examples of element selectors:

1. Styling Paragraphs (<p> elements):

```
/* Selects all paragraphs and sets their color to blue */  
p {  
  color: blue;  
}
```

```
/* Selects paragraphs with a specific class and sets their font size */  
p.special {  
  font-size: 18px;  
}
```

2. Styling Headings (<h1>, <h2>, ..., <h6> elements):

```
/* Selects all headings and sets their text color to green */  
h1, h2, h3, h4, h5, h6 {  
  color: green;  
}
```

```
/* Selects a specific heading level and sets its font weight */  
h2 {  
  font-weight: bold;  
}
```

3. Styling Lists (, , elements):

```
/* Selects all unordered lists and sets their list-style type to square */  
ul {  
  list-style-type: square;  
}
```

```
/* Selects list items within ordered lists and sets their color */
```

```
ol li {  
    color: #FF6347;  
}
```

4. Styling Links (<a> elements):

```
/* Selects all links and sets their text decoration to none */  
a {  
    text-decoration: none;  
}  
  
/* Selects links with a specific class and sets their color */  
a.highlight {  
    color: #FFD700;  
}
```

5. Styling Tables (<table>, <tr>, <th>, <td> elements):

```
/* Selects all tables and sets their border-collapse property */  
table {  
    border-collapse: collapse;  
}  
  
/* Selects table headers and sets their background color */  
th {  
    background-color: #4CAF50;  
    color: white;  
}
```

These are just a few examples of how element selectors can be used to style different HTML elements. Element selectors are simple but powerful, allowing you to apply styles consistently across multiple instances of the same element throughout your HTML document.

- **Class selectors:** These selectors select all elements that have a specific class attribute. For example, the selector `.red` selects all elements that have the class `red`. Class selectors in CSS are used to apply styles to elements with a specific class attribute. Class selectors are preceded by a dot (.) followed by the class name. Here are some examples of class selectors:

1. Styling Elements with a Specific Class:

```
/* Selects all elements with class "highlight" and sets their background  
color */  
.highlight {  
    background-color: yellow;  
    color: black;  
}
```

```

/* Selects all elements with class "btn" and sets common styling */
.btn {
    padding: 10px;
    border: 1px solid #333;
    background-color: #008CBA;
    color: #fff;
}

```

2. Combining Class Selectors:

```

/* Selects elements with both classes "highlight" and "text" */
.highlight.text {
    font-weight: bold;
}

```

```

/* Selects all buttons with either class "btn" or "secondary" */
.btn, .secondary {
    font-size: 16px;
}

```

3. Specific Elements with a Class:

```

/* Selects only paragraphs with class "info" */
p.info {
    font-style: italic;
}

```

```

/* Selects only list items with class "important" */
li.important {
    color: red;
}

```

4. Nested Class Selectors:

```

/* Selects anchor elements with class "link" inside a div with class "container" */
.container a.link {
    text-decoration: underline;
}

```

5. Dynamic Class Selectors (Hover Effect):

```

/* Applies a different background color when hovering over elements with
class "hoverable" */
.hoverable:hover {
    background-color: #FFD700;
}

```

In these examples, class selectors are used to target specific elements with matching class attributes and apply styles accordingly. Classes provide a way to

selectively style elements based on their purpose or characteristics, promoting a modular and reusable approach to CSS styling.

Here is an example of Class Selector.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>CSS Class Selector Example</title>
</head>
<style>
.highlight {
  background-color: yellow;
  color: black;
}

.blueStyle {
  color: blue;
}

</style>
<body style="font-family: 'Arial', sans-serif; background-color: #f4f4f4;">

  <header class="header">
    <h1 class="header-title">CSS Class Selector Example</h1>
  </header>

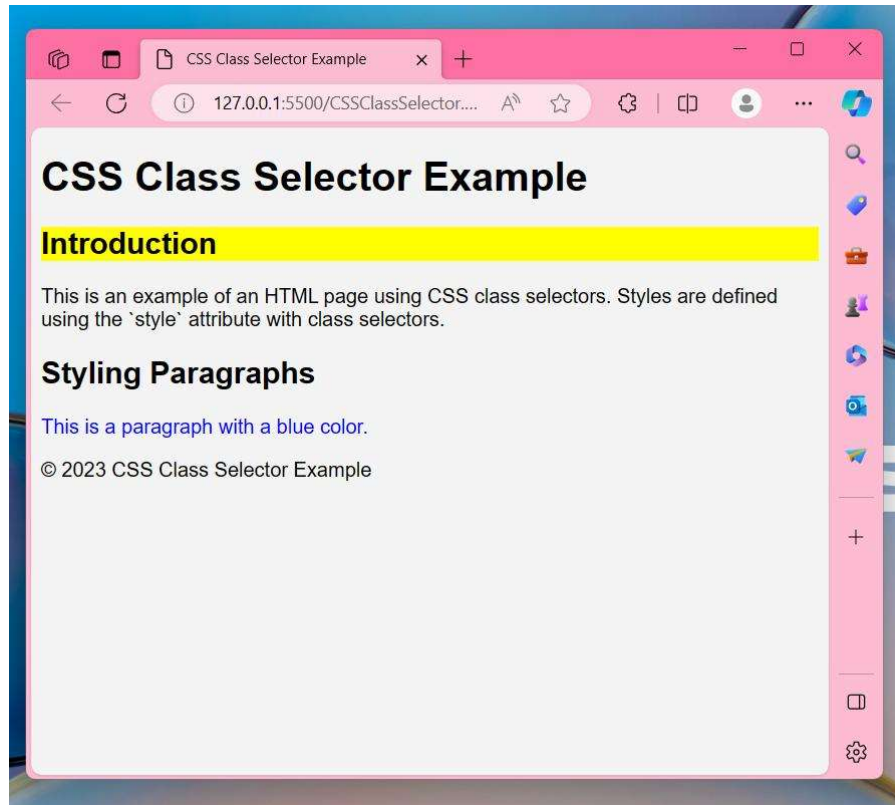
  <section class="main-section">
    <h2 class="highlight">Introduction</h2>
    <p class="section-text">This is an example of an HTML page using
CSS class selectors. Styles are defined using the `style` attribute with
class selectors.</p>

    <h2 class="section-heading">Styling Paragraphs</h2>
    <p class="blueStyle">This is a paragraph with a blue color.</p>

  <footer class="footer">
    <p class="footer-text">© 2023 CSS Class Selector Example</p>
  </footer>

</body>
</html>
```

If you open this in the browser it look like.



Let us see another example of class selector. This example demonstrates the class Selector for the specific HTML element.

```
<!DOCTYPE html>
<html>
<head>
  <title>class selector</title>
  <style>
    .selector {
      color: green;
      background-color: orange;
    }

    .center {
      background-color: yellow;
    }
  </style>
</head>

<body style="text-align: center">
```

```
<h1 class="selector">
    Class Selector Example
</h1>
<h2 class="center">This is a heading</h2>
<p class="selector center">
    Welcome to the IT industry: A computer science portal
</p>
</body>
</html>
```

Output:



CSS Properties

CSS properties are used to specify the style of an HTML element. There are many different CSS properties, which can be used to control the appearance, layout, and behavior of HTML elements. Some common CSS properties include:

Color: This property specifies the color of the text or background of an element.

Font-family: This property specifies the font family of the text of an element.

Font-size: This property specifies the font size of the text of an element.

Margin: This property specifies the margin around an element.

Padding: This property specifies the padding inside an element.

Position: This property specifies the position of an element relative to its normal position.

CSS Values

CSS values are used to specify the value of a CSS property. The value of a CSS property can be a number, a keyword, or a combination of both.

For example, the following CSS rule will set the margin of all paragraph elements to 10 pixels:

```
p {
    margin: 10px;
}
```

CSS Specificity

CSS specificity is a concept that determines which CSS rule should be applied to an element when multiple CSS rules apply to the element. The more specific a CSS rule is, the higher its specificity, and the more likely it is to be applied to the element.

There are four factors that determine the specificity of a CSS rule:

The number of ID selectors in the rule: ID selectors have the highest specificity.

The number of class selectors in the rule: Class selectors have lower specificity than ID selectors.

The number of element selectors in the rule: Element selectors have the lowest specificity.

The presence of inline styles: Inline styles have the highest specificity of all.

CSS Media Queries

CSS media queries are used to specify different styles for different media types, such as screens, printers, and projectors. For example, the following CSS media query will apply the specified styles to all screens that are less than 600 pixels wide:

```
@media (max-width: 600px) {  
  body {  
    font-size: 14px;  
  }  
}
```

CSS is a powerful and versatile language that can be used to create beautiful and dynamic Web pages. With a little practice, you can learn how to use CSS to create your own stunning Websites.

IX. Browser support for CSS

While building a Webpage using CSS, it is important to know the extent to which different Web browsers correctly interpret and display CSS (Cascading Style Sheets) code used to style Web pages. CSS is essential for controlling the layout, appearance, and formatting of Web content. However, not all CSS properties and features are supported uniformly across all Web browsers. This can lead to inconsistencies in how a Website appears to users, depending on the browser they are using.

When designing a Website, it's crucial to be aware of the CSS features that are supported by various browsers. Here are a few key points to consider regarding browser support for CSS:

1. Standardization:

CSS is standardized by the W3C (World Wide Web Consortium), but different browsers might interpret the specifications differently. As new CSS features are introduced, it takes time for browsers to adopt and fully support them.

2. Vendor Prefixes:

To address compatibility issues, browser vendors sometimes introduce experimental or non-standard CSS properties with vendor-specific prefixes. For example, `-webkit-` for Chrome and Safari, `-moz-` for Firefox, and `-ms-` for Internet Explorer/Edge. Developers can use these prefixes to target specific browsers, but it's important to note that they should also include the standard property without prefixes for better future compatibility.

Example:

```
.example {  
    -webkit-border-radius: 5px;  
    -moz-border-radius: 5px;  
    border-radius: 5px;  
}
```

3. Feature Detection:

Modern Web development practices involve feature detection techniques, such as using JavaScript libraries like Modernizr, to check if a browser supports a specific CSS feature before applying styles that depend on it. This ensures graceful degradation or progressive enhancement based on browser capabilities.

4. CSS Compatibility Tables:

Several online resources, such as caniuse.com, provide CSS compatibility tables. These tables indicate which CSS properties and features are supported by various browsers and their versions. Developers can consult these tables to make informed decisions about which CSS features to use based on the target audience's browser usage.

5. Polyfills:

For older browsers that lack support for certain CSS features, developers can use polyfills. Polyfills are JavaScript code that mimic the behavior of unsupported CSS features, allowing developers to create a consistent experience across different browsers.

6. Progressive Enhancement:

Adopting a progressive enhancement approach means building a basic, functional version of a Website that works across all browsers. Advanced CSS features can then be added for modern browsers that support them, providing an enhanced experience for users with capable browsers while ensuring a functional experience for all users.

In summary, understanding browser support for CSS is essential for creating cross-browser compatible Websites. Developers need to be aware of the various techniques available, such as feature detection, vendor prefixes, and polyfills, to handle CSS compatibility issues effectively and provide a seamless user experience across different Web browsers.

X. Different categories of CSS with Examples

CSS (Cascading Style Sheets) can be categorized into several types based on their usage and purpose.

Here are the different categories of CSS along with examples for each

- Inline CSS
- Embedded or Internal CSS
- External CSS

We have already learnt how to use Internal CSS with `<style>` tag. We will learn more about External CSS in the upcoming session.

XI. Inline CSS

Inline CSS is used to style a specific HTML element. For this CSS style, you'll only need to add the style attribute to each HTML tag, without using selectors. We already had a brief discussion on this in the previous session.

This CSS type is not always recommended, as each HTML tag needs to be styled individually. Managing your Website may become too hard if you only use inline CSS. However, inline CSS in HTML can be useful in some situations.

For example, in cases where you don't have access to CSS files or need to apply styles for a single element only. We will use the `.container` class to set the margins.

Let's first understand the use of the `'container'` class.

Containers

Containers are used to pad the content inside of them, and there are two container classes available:

1. The `.container` class provides a responsive fixed width container
2. The `.container-fluid` class provides a full width container, spanning the entire width of the viewport

Fixed Container

Use the `.container` class to create a responsive, fixed-width container.

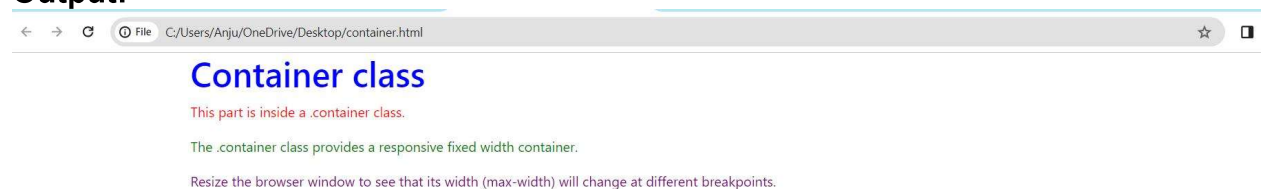
Note that its width (max-width) will change on different screen sizes:

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	Extra large ≥1200px
max-width	100%	540px	720px	960px	1140px

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Bootstrap Example</title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css">
<body>
  <!-- .container class -->
  <div class="container">
    <h1 style="color:blue">Container class</h1>
    <p style="color:red">This part is inside a .container class.</p>
    <p style="color:green">The .container class provides a responsive fixed width
container.</p>
    <p style="color:purple">Resize the browser window to see that its width (max-width)
will change at different breakpoints.</p>
  </div>
</body>
</html>
```

Output:



Fluid Container

Use the .container-fluid class to create a full width container, that will always span the entire width of the screen (width is always 100%):

Example:

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <title>Bootstrap Example</title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/css/bootstrap.min.css">
  </head>
<body>
  <!-- .container-fluid class -->
  <div class="container-fluid">
    <h1 style="color:blue">Container class</h1>
    <p style="color:red">This part is inside a .container-fluid class.</p>
    <p style="color:green">The .container-fluid class provides a full width container,
spanning the entire width of the viewport.</p>
  </div>
</body>
</html>

```

Output:



Let's take an example of a book gallery and create a web page using container class.

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bookswagon</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css">
</head>

<body>

  <!-- Navigation Bar -->
  <nav class="navbar navbar-expand-lg navbar-light bg-light ">
  <!--used to set basic margins-->
    <div class="container">

```



```

    <!-- anchor tag with inline css -->
    <a class="navbar-brand" style="color:blue" href="#">Bookswagon</a>
<button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarNav"
    aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse">
    <ul class="navbar-nav ml-auto">
        <li class="nav-item">
            <!-- anchor tag with inline css -->
            <a class="nav-link"" style="color:#239B56">Home</a>
        </li>
        <li class="nav-item">
            <!-- anchor tag with inline css -->
            <a class="nav-link"" style="color:#239B56">Books</a>
        </li>
        <li class="nav-item">
            <!-- anchor tag with inline css -->
            <a class="nav-link"" style="color:#239B56">Authors</a>
        </li>
    </ul>
</div>
</div>
</nav>
<!--use to set basic margins-->
<div class="container">
<!--h1 tag with inline style-->
    <h1 style=" padding-top: 20px; text-align: center; padding-bottom: 20px;
color:blue">Bookswagon</h1>

    <div class="row">
        <div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
            <div class="card h-100 text-center">
                
                <div class="card-body" style="display: flex !important; align-items:
stretch;">
                    <h5 class="card-title">Rich Dad Poor Dad</h5>
                    <p class="card-text">Robert T. Kiyosaki</p>
                </div>
            </div>
        </div>
    </div>

```

```
<div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
  <div class="card h-100 text-center">
    
    <div class="card-body" style="display: flex !important; align-items:
stretch;">
      <h5 class="card-title">Atomic Habits</h5>
      <p class="card-text">James Clear</p>
    </div>
  </div>
</div>
```

```
<div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
  <div class="card h-100 text-center">
    
    <div class="card-body" style="display: flex !important; align-items:
stretch;">
      <h5 class="card-title">A Brief History Of Time</h5>
      <p class="card-text">Stephen Hawking</p>
    </div>
  </div>
</div>
```

```
<div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
  <div class="card h-100 text-center">
    
    <div class="card-body d-flex" style="display: flex !important; align-items:
stretch;">
      <h5 class="card-title">12 Rules For Life</h5>
      <p class="card-text">Jordan B. Peterson</p>
    </div>
  </div>
</div>
```

```
<div class="col-book mb-4" style="width: 20%; padding: 0.5rem;">
  <div class="card h-100 text-center">
```

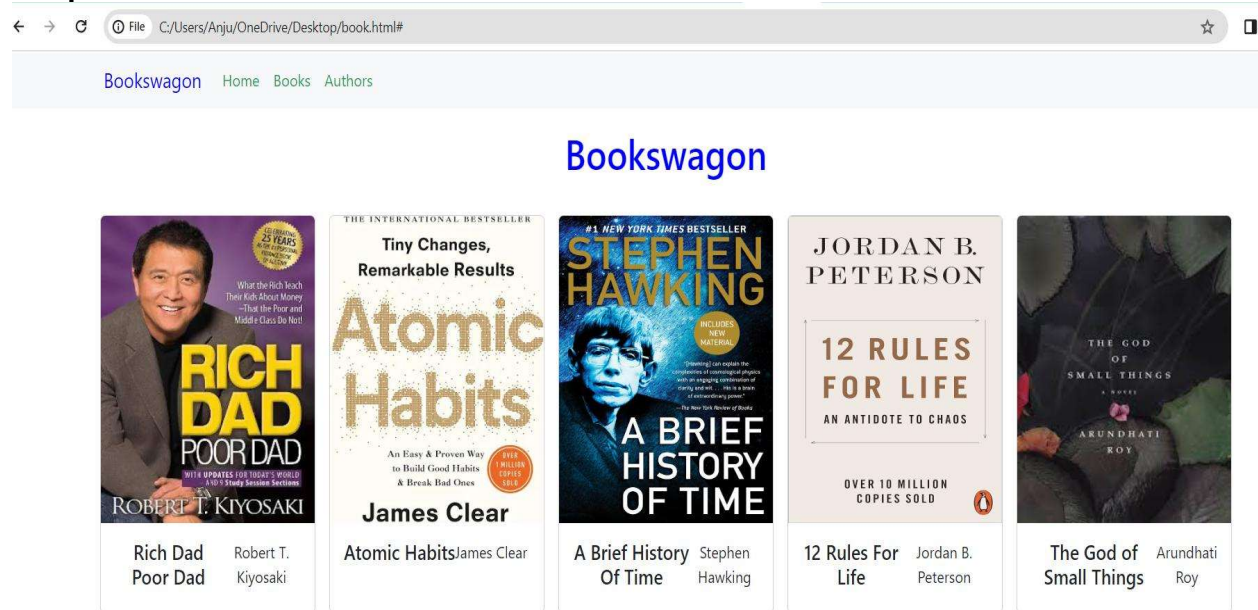
```


<div class="card-body" style="display: flex !important; align-items:
stretch;">
    <h5 class="card-title">The God of Small Things</h5>
    <p class="card-text">Arundhati Roy</p>
</div>
</div>
</div>
</div>
</div>
</body>

</html>

```

Output:



Advantages of Inline CSS:

You can easily and quickly insert CSS rules into an HTML page. That's why this method is useful for testing or previewing the changes and performing quick fixes to your Website.

You don't need to create and upload a separate document as in the external style.

Disadvantages of Inline CSS:

Adding CSS rules to every HTML element is time-consuming and makes your HTML structure messy.

Styling multiple elements can affect your page's size and download time.

Other types of CSS will be discussed in the next session.

ChatGPT Exercise:

Please visit the site - <https://chat.openai.com/> and create your account if not yet created. Next, start with the first question on the prompt:

I am a beginner in HTML and CSS, sometimes I am not able to load a full page of code and then run it. I am also getting lots of errors while running the page. Could you give tips on how to create an error-free HTML page with CSS that loads faster.

I am a beginner in HTML and CSS, sometimes I am not able to load a full page of code and then run it. I am also getting lots of errors while running the page. Could you give tips on how to create an error-free HTML page with CSS that loads faster.



ChatGPT can make mistakes. Consider checking important information.

In addition to the above question you can add any specific error you are getting. Submit the query clicking the green arrow icon. Now go through the response received from the AI engine. Output error will be something like the following screenshot.



Certainly! Creating error-free HTML and CSS code that loads quickly is essential for building efficient and user-friendly websites. Here are some tips to help you create a well-optimized web page:



1. Organize Your Code:

- Use proper indentation and formatting to make your code readable.
- Use comments (`<!-- comment -->` in HTML, `/* comment */` in CSS) to annotate your code and explain complex sections.

2. Validate Your HTML and CSS:

- Use online validators like W3C Markup Validation Service for HTML (<https://validator.w3.org/>) and W3C CSS Validation Service (<https://jigsaw.w3.org/css-validator/>) to check your code for errors.

3. Optimize Images:

- Compress images before using them on your website. Tools like TinyPNG (<https://tinypng.com/>) can reduce image file sizes without significant loss of quality.

Regenerate

Send a message



After the 2 hr ILT, the student has to do a 1 hour live lab. Please refer to the Session 5 Lab doc in the LMS.